Enterprate Limited

Build: Viability Engine

Objective

Create a deterministic **Viability Engine** that answers:

“Does this business model or service idea make commercial sense?”

It reads from:

- Commercial data
- Financial data
- Demand evidence
- Capacity data

It writes structured output for:

- Master Aggregator
- Fragility Index
- Scenario Simulation
- Adaptive Scenario Intelligence
- AI Narratives
- PDF Reports

Recommended Folder

/src/lib/engines/viability

Suggested structure:

```
/src/lib/engines/viability
  ├── types.ts
  ├── rules.ts
  ├── viability-engine.ts
```

```
├── viability-engine.test.ts
└── index.ts
```

1. Input Contract

```ts
export interface ViabilityEngineInput {
  businessId: string;

  revenue: {
    pricePerSale: number;
    expectedSalesPerMonth: number;
  };

  costs: {
    directLabourCostPerSale: number;
    contractorCostPerSale?: number;
    materialsCostPerSale?: number;
    travelCostPerSale?: number;
    otherDirectCostPerSale?: number;

    monthlySoftwareCost: number;
    monthlyMarketingCost: number;
    monthlyAdminCost: number;
    monthlyRentCost?: number;
    monthlyOtherFixedCost?: number;
  };

  capacity: {
    hoursRequiredPerSale: number;
    availableDeliveryHoursPerMonth: number;
  };

  demand: {
    demandEvidenceType:
      | "assumption_only"
      | "market_research"
      | "enquiries"
      | "LOIs"
```

```
      | "paid_pilot"
      | "paying_customers"
      | "repeat_paying_customers";

    numberOfInterestedLeads?: number;
    numberOfPayingCustomers?: number;
  };

  metadata?: {
    source?: "manual" | "imported" | "integration";
    confidenceLevel?: "LOW" | "MEDIUM" | "HIGH";
    rulesetVersion?: string;
  };
}
```

2. Output Contract

```
export interface ViabilityEngineOutput {
  engine: "viability";
  version: string;

businessId: string;

  metrics: {
    monthlyRevenue: number;
    variableCostPerSale: number;
    monthlyVariableCost: number;
    monthlyFixedCost: number;
    contributionPerSale: number;
    contributionMargin: number;
    breakEvenSales: number | null;
    breakEvenMonths: number | null;
    capacitySalesPerMonth: number | null;
    capacityUtilisation: number | null;
    capacityFeasible: boolean;
  };
```

```
scores: {
  marginScore: number;
  breakEvenScore: number;
  demandScore: number;
  capacityScore: number;
  viabilityScore: number;
};

classification: {
  outcome: "VERY_STRONG" | "STRONG" | "FAIR" | "WEAK";
  label: string;
};

riskFlags: Array<{
  code: string;
  severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
  explanation: string;
  recommendedAction: string;
  priorityScore: number;
}>;

recommendations: Array<{
  type: string;
  priority: "LOW" | "MEDIUM" | "HIGH";
  text: string;
  expectedImpact?: string;
}>;

interpretation: {
  summary: string;
  keyDriver: string;
  recommendation: string;
};

confidence: {
  level: "LOW" | "MEDIUM" | "HIGH";
  note: string;
};
```

```
  trace: Record<string, unknown>;
}
```

3. Calculations

Use the shared Computation Library.

Monthly Revenue

```
monthly_revenue = price_per_sale × expected_sales_per_month
```

Variable Cost Per Sale

```
variable_cost_per_sale =
direct_labour_cost_per_sale +
contractor_cost_per_sale +
materials_cost_per_sale +
travel_cost_per_sale +
other_direct_cost_per_sale
```

Monthly Variable Cost

```
monthly_variable_cost =
variable_cost_per_sale × expected_sales_per_month
```

Monthly Fixed Cost

```
monthly_fixed_cost =
monthly_software_cost +
monthly_marketing_cost +
monthly_admin_cost +
monthly_rent_cost +
monthly_other_fixed_cost
```

Enterprate Limited

Contribution Per Sale

```
contribution_per_sale =
price_per_sale - variable_cost_per_sale
```

Contribution Margin

```
contribution_margin =
(monthly_revenue - monthly_variable_cost) / monthly_revenue
```

Break-even Sales

```
break_even_sales =
monthly_fixed_cost / contribution_per_sale
```

If contribution_per_sale <= 0, return null.

Break-even Months

```
break_even_months =
break_even_sales / expected_sales_per_month
```

If break_even_sales is null or expected_sales_per_month <= 0, return null.

Capacity Sales Per Month

```
capacity_sales_per_month =
available_delivery_hours_per_month / hours_required_per_sale
```

Capacity Utilisation

```
capacity_utilisation =
expected_sales_per_month / capacity_sales_per_month
```

4. Scoring Rules

Margin Score

```
if contributionMargin > 0.5 => 100
else if contributionMargin >= 0.3 => 75
else if contributionMargin >= 0.15 => 50
else => 20
```

Break-even Score

```
if breakEvenMonths === null => 20
else if breakEvenMonths <= 6 => 100
else if breakEvenMonths <= 12 => 75
else if breakEvenMonths <= 24 => 50
else => 20
```

Demand Score

```
const demandScoreMap = {
  repeat_paying_customers: 100,
  paying_customers: 85,
  paid_pilot: 70,
  LOIs: 55,
  enquiries: 40,
  market_research: 30,
  assumption_only: 15
};
```

Boosts:

```
if numberOfPayingCustomers >= 5 => +5
if numberOfInterestedLeads >= 20 => +5
cap at 100
```

Capacity Score

```
if capacityFeasible === false => 20
else if capacityUtilisation >= 0.5 && capacityUtilisation <= 0.8 =>
100
else if capacityUtilisation > 0.8 && capacityUtilisation <= 1 => 70
else if capacityUtilisation < 0.5 => 60
else => 20
```

5. Final Viability Score

```
viability_score =
0.35 × margin_score +
0.25 × break_even_score +
0.25 × demand_score +
0.15 × capacity_score
```

6. Outcome Logic

```
if viabilityScore >= 85 => VERY_STRONG
else if viabilityScore >= 70 => STRONG
else if viabilityScore >= 50 => FAIR
else => WEAK
```

Labels:

```
VERY_STRONG = "Very Strong"
STRONG = "Strong"
FAIR = "Fair"
WEAK = "Weak"
```

7. Risk Flags

LOW_MARGIN_RISK

Trigger:

```
contributionMargin < 0.3
```

Severity:

```
contributionMargin < 0.15 ? "HIGH" : "MEDIUM"
```

Action:

```
Review pricing, direct delivery cost, supplier cost, or service scope.
```

NO_BREAK_EVEN_RISK

Trigger:

```
breakEvenSales === null
```

Severity:

```
CRITICAL
```

Action:

```
Redesign pricing or cost structure before launch.
```

SLOW_PAYBACK_RISK

Trigger:

```
breakEvenMonths !== null && breakEvenMonths > 12
```

Severity:

```
breakEvenMonths > 24 ? "HIGH" : "MEDIUM"
```

Action:

Reduce fixed costs, improve contribution margin, or increase sales velocity.

UNPROVEN_DEMAND_RISK

Trigger:

```
demandScore <= 30
```

Severity:

HIGH

Action:

Secure paid pilots, pre-orders, LOIs, or paying customers before scaling.

CAPACITY_CONSTRAINT_RISK

Trigger:

```
capacityFeasible === false
```

Severity:

HIGH

Action:

Increase delivery capacity, reduce expected sales target, or improve delivery efficiency.

OVERLOAD_RISK

Trigger:

```
capacityUtilisation !== null && capacityUtilisation > 0.9
```

Severity:

```
MEDIUM
```

Action:

```
Monitor delivery load and avoid scaling beyond current capacity.
```

8. Recommendations Logic

Generate top 3 recommendations based on risks.

Priority order:

1. NO_BREAK_EVEN_RISK
2. CAPACITY_CONSTRAINT_RISK
3. LOW_MARGIN_RISK
4. UNPROVEN_DEMAND_RISK
5. SLOW_PAYBACK_RISK
6. OVERLOAD_RISK

Example recommendations:

```
[
  {
    type: "pricing_optimization",
    priority: "HIGH",
    text: "Increase price or reduce direct cost to improve
contribution margin."
  },
  {
    type: "demand_validation",
    priority: "HIGH",
```

```
    text: "Validate demand with paid pilots or signed intent before
scaling."
  }
]
```

9. Interpretation Rules

VERY_STRONG

The service idea shows strong economics, credible demand, and feasible delivery capacity.

STRONG

The service idea is viable, but one area should be improved before scaling aggressively.

FAIR

The service idea may work, but it has important weaknesses that should be corrected before major investment.

WEAK

The service idea is currently not attractive enough to launch confidently without redesign.

10. Confidence Logic

```
if metadata.confidenceLevel exists:
  use it
else if demandEvidenceType includes paying customers or paid pilot:
  MEDIUM
```

```
else:
  LOW
```

Confidence notes:

```
HIGH: Based on verified or integrated business data.
MEDIUM: Based on partial evidence and structured assumptions.
LOW: Based mainly on estimates and should be validated with real
market data.
```

11. Unit Tests

Create tests for:

Very Strong Case

- margin > 50%
- break-even < 6 months
- paid pilot
- capacity feasible

Expected:

- outcome VERY_STRONG
- no critical risks

Weak Case

- margin < 15%
- no break-even
- assumption only
- capacity not feasible

Expected:

- outcome WEAK

- risk flags include LOW_MARGIN_RISK, NO_BREAK_EVEN_RISK, UNPROVEN_DEMAND_RISK, CAPACITY_CONSTRAINT_RISK

Capacity Constraint Case

- expected sales greater than capacity

Expected:

- CAPACITY_CONSTRAINT_RISK

Demand Boost Case

- paying customers >= 5
- interested leads >= 20

Expected:

- demand score boosted but capped at 100

Break-even Null Case

- contribution per sale <= 0

Expected:

- breakEvenSales null
- breakEvenMonths null
- NO_BREAK_EVEN_RISK

12. Acceptance Criteria

Codex should complete this when:

- Viability Engine accepts the defined input contract
- It uses computation library functions

- It returns the defined output contract
- All risk flags are generated correctly
- Recommendations are prioritised
- Interpretation is deterministic
- Confidence is included
- Unit tests pass
- No database calls inside engine
- No LLM calls inside engine

13. Codex Prompt

```
Build the EnterprateAI Viability Engine in /src/lib/engines/viability.

The engine must be a pure deterministic TypeScript module. It must not
call the database, API, or LLM.

It should accept ViabilityEngineInput and return
ViabilityEngineOutput.

Use the shared computation library from /src/lib/computation for:
- contribution margin
- contribution per unit
- break-even units
- break-even months
- capacity sales per month
- capacity utilisation
- capacity feasibility
- weighted score

Implement:
- margin score
- break-even score
- demand score
- capacity score
- final viability score
- outcome classification
```

- risk flags
- top 3 recommendations
- interpretation
- confidence layer
- trace object

Create:
- types.ts
- rules.ts
- viability-engine.ts
- viability-engine.test.ts
- index.ts

Add unit tests for:
- very strong case
- weak case
- capacity constraint
- demand boost
- no break-even case

Export the engine from index.ts.